



adbox
TECHNOLOGIES

Fleet Dashboard

Operator console: what each page does and how it reaches the API.



Contents

How it talks to the API 3

Pages 3

Things worth knowing 4

Building 4

Structure 4

adbox-dashboard

The operator console for the adbox fleet. Next.js 16, App Router.

```
npm install
cp .env.example .env.local      # point at adbox-server, set the admin key
npm run dev                     # http://localhost:3000
```

Variable	Purpose
ADBOXAPIORIGIN	Base URL of adbox-server, e.g. <code>http://localhost:8788</code> .
ADBOXADMINAPIKEY	Must match ADMINAPIKEY in <code>adbox-server/.env</code> .

How it talks to the API

Every request goes through `app/api/[...path]/route.js`, a server-side proxy that attaches `x-admin-key`. The admin credential is therefore **never** present in a client bundle, and the proxy only allow-lists the admin routes — a browser cannot reach `/api/device/*` through it even if it tries.

Client code calls the thin helpers in `app/lib/api.js`; no component builds URLs by hand.

Pages

Route	What it is for
<code>/</code>	Fleet health at a glance: online/offline, how many boxes have today's playlist, active campaigns, 24 h errors, a live map and the most recent schedule handovers.
<code>/fleet</code>	Every registered box, searchable, with connectivity and last known position.
<code>/fleet/[id]</code>	One box in depth — map, hardware, schedule preview (a dry run of the resolver for any date), delivery history with raw payloads, proof of play, errors, and settings including API key rotation.
<code>/provisioning</code>	Issue and revoke single-use codes, with the exact installer command to run on the Pi.
<code>/campaigns</code>	Create and edit campaigns: playlist, date range, days of week, daily window, priority and target (boundary, specific boxes, or the whole fleet).
<code>/media</code>	Upload images and video, set default durations, delete unused items. Duplicate uploads are detected by checksum.
<code>/boundaries</code>	Draw polygon or radius boundaries on a map. These are what geo-targeted campaigns match against.
<code>/telemetry</code>	Playback, schedule deliveries and errors across the fleet, filtered by service date.

Things worth knowing

Schedule preview is the debugging tool. When someone asks why a screen is blank, open the box, pick the date, and look at what the resolver actually returns. It runs the same code path as the device endpoint.

Empty deliveries are flagged. A delivery with 0 items is shown as a warning badge rather than a zero, because it means a box asked and got nothing.

Fallback positions are visible. A box with no GPS fix that is using configured coordinates is tagged **fallback** everywhere its position appears, so mis-targeted ads are obvious rather than silent.

Rotating a key breaks the box until you apply it. The dashboard shows the new key once, along with the two commands to run on the Pi.

Retire, do not delete. Setting a box to **retired** makes the device API reject it, which is what you want for a stolen or decommissioned unit. Deleting the record discards its history.

Building

```
npm run build
npm start
```

Deploy behind the same TLS terminator as the API, and put your own authentication in front of it — the dashboard assumes whoever reaches it is already authorised.

Structure

```
app/
+-- api/[...path]/route.js    Authenticated proxy (server only)
+-- lib/api.js                Client helpers and formatters
+-- components/
|   +-- Sidebar.jsx           Navigation with live fleet counts
|   +-- ui.jsx                 Panel, Stat, Badge, Empty, Modal, useResource, useToast
|   +-- FleetMap.jsx          Read-only Leaflet map
|   +-- BoundaryEditor.jsx     Drawing surface for boundaries
+-- globals.css
+-- <route>/page.jsx
```